

Roteiro da Aula 4: servidor, Docker, Kamal e deploy

Deck: `slides/build/aula-04-servidor-docker-kamal-e-deploy.pptx` (62 slides, sendo 7 de prática)
Antes de tudo: `guia-do-instrutor.md`, conduzir a sala, não o conteúdo **Apostila da turma:** `04-deploy.md` · **Checkpoint:** `aula-04` **Apêndice:** `apendice-azure.md`, o mesmo percurso na nuvem, para casa

Esta era a aula mais arriscada das quatro, porque o gargalo não era digitar: era clicar em portal, e portal falha. **Agora o servidor é a própria máquina de cada aluno.** Não há VM para criar, nem rede para configurar, nem nuvem para esperar. Sobrou o que realmente ensina.

O que preparar com antecedência

#	O que	Quando
1	Cobrar de todo mundo a saída de <code>ssh "\$USER"@127.0.0.1 'echo ok'</code>	uma semana antes
2	Um SMTP (Resend ou outro) com domínio verificado, e uma key para a turma	véspera
3	Fazer o percurso inteiro sozinho, num usuário limpo, do <code>apt install openssh-server</code> ao <code>curl --cacert</code>	véspera

O item 1 é o que tira o risco do dia

O Kamal precisa de duas coisas na máquina: **SSH que aceite chave** e **Docker rodando**. O Docker eles têm desde a Aula 1. O SSH é o que pode faltar, e descobrir isso em aula custa vinte minutos.

Peça no grupo, uma semana antes:

```
sudo apt install -y openssh-server && sudo service ssh start      # WSL2, Ubuntu, Debian
ssh "$USER"@127.0.0.1 'echo FUNCIONOU'                          # print disto
```

No macOS não se instala nada: **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

O tropeço que vai aparecer: no WSL2 o `sshd` não sobe sozinho a cada boot do Windows. Quem reiniciou entre o teste e a aula vai chegar com `Connection refused`. A saída é uma linha (`sudo service ssh start`), e vale você avisar de véspera para ninguém perder tempo com isso.

Antes de começar

- [] Dois terminais grandes. Hoje **os dois são a mesma máquina**, e vale dizer isso: um é onde você edita e roda o Kamal, o outro é onde você olha `docker ps` e logs.
- [] Um navegador aberto, para o momento do aviso de certificado.
- [] `docs/troubleshooting.md` aberto numa aba.
- [] Portal da Azure logado numa aba anônima, para a demo do fim (não vaze recursos do SEEM na projeção).
- [] O PAT do ghcr.io: avise no grupo, na véspera, para já virem com ele criado.
- [] O seu próprio percurso refeito na véspera, com o `~/ssh/capacita` apagado, para você fazer do zero junto com eles.

Cronograma

As sete práticas são o esqueleto do dia. Estão em negrito, e nesta aula elas são quase tudo: o conteúdo existe para explicar o que a mão está fazendo.

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura, e de onde paramos	6
00:06	4–10	O problema, as opções, e por que a máquina é a sua	18
00:24	11–18	Chaves, SSH, e autorizar a si mesmo	20
00:44	19	Prática 1: a sua máquina vira o servidor	20
01:04	20–25	Docker: imagem, Dockerfile, registry, arquitetura	16
01:20	26	Prática 2: arquivos de deploy e o token	15
01:35	—	Intervalo	10
01:45	27–35	Kamal: <code>deploy.yml</code> , accessory, segredos	22
02:07	36	Prática 3: o <code>.env</code>	15
02:22	37–44	TLS, certificado, CA, proxy reverso	22
02:44	45	Prática 4: certificado e <code>/etc/hosts</code>	15
02:59	46–50	CI, o primeiro deploy, operar, backup	14
03:13	51–52	Práticas 5 e 6: o deploy, e o certificado com os olhos	35
03:48	53–56	Num servidor de verdade: demo	15
04:03	57	Prática 7: o sistema inteiro, e o backup	20
04:23	58–62	O que vocês fizeram, e agora, fim	8

Dá 4h31. Caiu ~45 minutos em relação à versão com VM, e o que saiu foi exatamente o que não ensinava back-end: criar máquina, esperar download de imagem e configurar rede.

Corte **nesta ordem**:

#	O que cortar	Ganho
1	Prática 7 vira dever de casa (fluxo da Aula 3 em produção, e o backup)	-20
2	A demo de servidor de verdade (53-56) encolhe para 6 min: só os slides 54 e 55	-9
3	Slides 22-23 (Dockerfile e as duas decisões) ficam na apostila	-8
4	Slides 34-35 (credentials × ENV, três camadas) ficam na apostila	-8
5	Prática 4: você gera um certificado e distribui; eles só fazem o <code>/etc/hosts</code>	-8
6	Prática 2: mande fazer o <code>rsync</code> e o PAT de casa , na véspera	-12

Cortando de 1 a 3, fecha em **3h54**. Cortando os seis, **3h18**.

Nunca corte as Práticas 5 e 6. Ver a própria API no ar e entender o aviso de certificado são os dois momentos que a turma leva embora.

Onde o tempo ainda pode escapar

Risco	Sinal	O que fazer
<code>sshd</code> parado	<code>Connection refused</code> na Prática 1	<code>sudo service ssh start</code> . É uma linha, e é o erro mais comum do dia
Permissão de chave	<code>Permission denied (publickey)</code>	<code>chmod 700 ~/.ssh</code> e <code>chmod 600</code> na chave e no <code>authorized_keys</code>
Build da imagem	<code>kamal setup</code> demorando	Normal na primeira vez. Use o tempo para o slide de backup e para perguntas
Arquitetura errada	<code>exec format error</code> no container	<code>uname -m</code> . Quem tem Mac com chip M precisa de <code>arm64</code>

Plano B, decidido de véspera

B1: cortar o TLS. Se às 02:22 a maioria ainda não deployou, tire `proxy.ssl` do `deploy.yml`, suba em HTTP puro, e faça o bloco de TLS todo projetado, no seu. **Devolve ~20 min.** Eles refazem em casa com a apostila.

B2: virar demo a partir do intervalo. Se às 01:35 não houver pelo menos metade da turma com o `SSH_OK` na tela, pare de esperar e projete o percurso do começo ao fim.

Bloco a bloco

00:06 · O problema e as opções (4–10)

Comece pelo slide 5, que é a pergunta honesta: `bin/rails server` **funciona enquanto o terminal está aberto — e daí?** As cinco necessidades listadas ali são a aula inteira, e nenhuma depende de onde a máquina está.

O slide 9 é o que dá credibilidade: admita o custo. Uma máquina só é ponto único de falha, volume não é backup, o Postgres não é gerenciado.

O **10** é o slide que define o dia:

"O Kamal não sabe onde a máquina está. Ele conecta por SSH e roda `docker` do outro lado. Se o endereço for 127.0.0.1, ele conecta na de vocês."

Diga com todas as letras que não é faz de conta. Alguém vai desconfiar, e com razão: é a mesma conexão SSH, o mesmo `deploy.yml`, os mesmos comandos. O que um servidor alugado acrescenta são duas linhas do `.env` e o trabalho de provisionar a máquina, que está no apêndice.

00:24 · Chaves e SSH (11–18)

- **12** — o Kamal precisa de duas coisas, e uma delas eles já têm. Isso desarma a sensação de que hoje é tudo novo.
- **13–14** — chave pública e privada. É o conceito que também explica o JWT da Aula 3 e o TLS que vem daqui a pouco. **Não pule**, mesmo com a turma ansiosa para digitar.
- **15** — instalar o `ssh`. **Avise aqui, antes de alguém apanhar:** no WSL2 ele não sobe sozinho no boot.
- **16** — autorizar a si mesmo. Vale a pausa: *"isto é literalmente o que vocês fariam num servidor alugado. A mesma chave, o mesmo arquivo, o mesmo comando. Muda o endereço."*
- **17** — o `chmod 600`. Diga que o SSH **recusa** chave com permissão frouxa, e que é a primeira coisa a conferir no `Permission denied (publickey)`.

Ponto de decisão às 01:04: se menos da metade tiver o `SSH_OK`, resolva junto antes de seguir. Sem isso, nada depois acontece.

01:04 · Docker (20–25)

Conceitual e rápido. Imagem é receita, container é bolo.

No **23**, as duas decisões: multi-stage (compilador não vai para produção) e usuário não-root (*"é uma linha que muda o tamanho do estrago"*).

O **24** explica por que a imagem sobe para o ghcr.io se quem builda e quem roda são a mesma máquina. **Alguém vai perguntar, e a pergunta é boa.** A resposta: é o que aconteceria com um servidor do outro lado do mundo, e é o que faz o mesmo `deploy.yml` servir nos dois casos sem mudar uma linha.

O **25** é operacional: PAT com `write:packages`, e a arquitetura. **Circule pela sala aqui:** quem tem Mac com chip M precisa de `arm64`, e quem errar isso só descobre no meio do `kamal setup`, com uma mensagem que não diz "arquitetura".

01:45 · Kamal (27–35)

Abra o `config/deploy.yml` projetado e percorra junto com os slides.

O slide 30 (`clear` × `secret`) tem o argumento concreto: `JWT_SECRET` no `clear` apareceria em `docker inspect` e no histórico do shell. Se der, mostre um `docker inspect` de verdade.

O **34** (três camadas de segredo) é o slide que amarra tudo. Deixe na tela enquanto explica, e diga a linha que conecta com o apêndice: *"quando isso virar um pipeline, só a primeira camada muda."*

No **35**, pare no `.env`: `git status` **não pode mostrar esse arquivo.** Mande todo mundo rodar, ali, na frente de você.

02:22 · TLS (37–44)

O melhor bloco da aula.

- **38–41** — HTTPS é HTTP num túnel; handshake em três passos; certificado e cadeia de confiança.
- **41** é a virada: *"para uma CA pública assinar, você tem que provar que o domínio é seu. E vocês não têm domínio nenhum."* É daí que sai o autoassinado, como consequência e não como gambiarra.
- **43** — `.test` e `/etc/hosts`. Vale a curiosidade: `.test` é reservado por RFC exatamente para isso, e ninguém consegue registrar.
- **44** — **o slide da aula.** Não passe rápido.

03:13 · O deploy, e o certificado (51–52)

```
source .env && bundle exec kamal config
```

Valida sem tocar em nada. **Faça isso antes:** pega variável faltando de graça.

Depois `kamal setup`. Use a espera do build para o slide de backup e para perguntas.

E então, o momento da aula. Diga o que está na frente deles, porque nem todo mundo percebe sozinho:

"O código de vocês está rodando dentro de um container, em modo produção, atrás de um proxy que termina TLS, com um Postgres que tem volume. Vocês não estão mais rodando `bin/rails server`."

Na Prática 6, os três `curl` respondem, um a um, o que TLS é e o que CA é:

```
curl https://seunome.test/...      # falha: self signed certificate
curl -k https://seunome.test/...    # funciona, sem conferir nada
curl --cacert tls/capacita-cert.pem ... # funciona, CONFERINDO
```

A frase que fixa: "o `-k` desliga a conferência. O `--cacert` não desliga nada, ele diz em quem confiar. Uma CA é isso e só isso: alguém em quem o seu sistema já decidiu confiar, de fábrica."

Depois abra no navegador e leia o aviso vermelho junto com eles. **É o mesmo fato, dito para um humano.**

03:48 · Num servidor de verdade, demo (53–56)

Quinze minutos, projetado, sem ninguém acompanhando no teclado. Diga isso antes de começar, ou metade da turma vai tentar criar conta na Azure e perder o fim da aula.

O slide 54 é o que importa: no `.env`, **duas linhas**. O resto do trabalho é provisionar a máquina, e é isso que você percorre no portal: VM → usuário → `apt upgrade` → Docker → firewall → `sshd` → endurecido → DNS → certificado → OIDC.

Não detalhe nenhum. O que você quer é que reconheçam as peças e saibam que o passo a passo existe.

Feche apontando o apêndice: `docs/apendice-azure.md`, e a **Azure for Students dá US\$100 sem cartão**.

04:23 · Prática 7 e fecho (57–62)

O slide 59 (`O QUE O SEEM-BACKEND TEM A MAIS`) é o convite: *"nada disso é difícil depois do que vocês viram hoje. O código está lá, e agora vocês conseguem ler."*

E então **reserve cinco minutos de verdade para os slides 60 e 61**. Não corra.

O **60** (`O QUE VOCÊS FIZERAM`) é a lista do que construíram, e existe porque quem fez raramente percebe o tamanho: o esforço esconde a conquista. Leia devagar, olhando para eles.

O **61** (`E AGORA?`) deixa o canal aberto. Diga em voz alta que dúvida daqui a três semanas ainda é dúvida: a pergunta que chega depois costuma ser a mais importante da capacitação inteira, porque é a primeira que veio de um problema real deles.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Isso não é 'de verdade', é só na minha máquina."	É de verdade: é o mesmo Docker, o mesmo Kamal, o mesmo <code>deploy.yml</code> , a mesma conexão SSH. O que falta é o IP ser público, e são duas linhas no <code>.env</code> .
"Então por que não usar a nuvem direto?"	Porque metade da aula viraria espera de portal e de cota, e nada disso é back-end. O apêndice tem o caminho, e vocês já vão saber operar quando chegarem lá.
"Por que a imagem vai para o GitHub se roda aqui mesmo?"	Porque é o que aconteceria com um servidor remoto, e é o que faz o mesmo arquivo servir nos dois casos. Também é o que te dá versão e rollback de graça.
"Por que não Heroku/Render/Vercel? É mais fácil."	É mesmo, e para muita coisa é a escolha certa. Aqui o objetivo é entender o que essas plataformas fazem por vocês, e o custo delas cresce rápido.
"Por que não rodar só com Docker Compose?"	Compose sobe container. Não resolve registry, build versionado, zero-downtime, rollback nem gestão de segredo. É o que o Kamal acrescenta.
"Meu navegador diz que o site não é seguro."	E está certo: o certificado é seu, assinado por você. É a Prática 6, e é a diferença entre criptografia e confiança.
"E se eu quiser desligar isso depois?"	<code>kamal app stop</code> e <code>kamal accessory stop db</code> . Para apagar de vez, <code>kamal remove</code> .
"Posso usar isso num projeto meu?"	Pode, e é a intenção. Troque o nome do serviço, o host e os segredos. O resto é igual.

Depois da capacitação

Mande no grupo:

1. `kamal app stop` e `kamal accessory stop db` para quem não quiser os containers rodando.
2. `apendice-azure.md`, com o link da Azure for Students, para quem quiser o IP público e o deploy automático.
3. O link do `seem-backend`, com o convite do slide 59.
4. `docs/troubleshooting.md`, que é o que eles vão abrir quando algo quebrar sem você por perto.